

Memtrace × BMAD — Integration Guide

⚡ Before You Start — Model Recommendations

This guide has two very different usage profiles. Use the right model for each:

Moment	Model	Why
Guided setup (Section 3 — interactive installation)	🧠 POWERFUL model + max thinking (Pro, Opus, Sonnet with reasoning enabled)	You'll make ~15 integration decisions that affect ALL your workflows. The AI needs to retain long context, navigate between Memtrace and BMAD tools, understand tradeoffs, and generate 9 structured <code>.toml</code> files. Flash/Haiku can lose the thread and forget answers mid-flow.
Daily use (<code>/bmad-code-review</code> , <code>/bmad-agent-dev</code> , etc.)	⚡ LIGHTWEIGHT model (Flash, Haiku, normal Sonnet)	The decisions are already in the <code>.toml</code> files. The agent just needs to follow the script — executing <code>get_impact</code> , <code>find_code</code> , <code>get_evolution</code> as instructed by <code>activation_steps_prepend</code> . Individual tool calls, no deep reasoning needed.

💡 **Rule of thumb:** If you're **INSTALLING** (setup), use the big brain. If you're **USING** (daily work), the lightweight one is enough.

Technical requirements: - BMAD installed in the project (with `_bmad/` at the root) - Memtrace MCP server running with the repository indexed (`memtrace index`) - AI client with MCP tool support (Claude Code, OpenCode, Cursor, etc.) - No tool-specific dependencies — the system is AI-client agnostic

Index

- [1. Case Study: Memtrace Impact on a BMAD Project](#)
- [2. Memtrace Tools Summary](#)
- [3. Integration by BMAD Workflow \(with agent and prompt\)](#)
- [4. Complete Prompts by Workflow](#)
- [5. AI-Guided Setup — Interactive Installation](#)



Case Study: Memtrace Impact on a BMAD Project

Context: Web project with ~280 indexed symbols, 5 logical modules, Next.js + PostgreSQL stack. A 5-story epic delivered the product's core intelligence engine (rule engine, reactive calendar, granular privacy). Timeline: 7 days.

Before × After: What Memtrace Changed in the Retrospective

Memtrace Tool	Concrete Result	Without Memtrace	Approximate Time Saved
<code>get_codebase_briefing</code>	Scope quantified in 5s: 485 new symbols, 2,273 edges, 5 modules	Read package.json, <code>tree</code> the project, open 5-10 files manually, infer architecture	~20 min → 5s
<code>get_evolution(compound)</code>	142 change episodes, top 10 most-changed files, 485 symbols created	<code>git log --oneline</code> , <code>git diff --stat</code> , manually count, miss branch-level changes	~30 min → 5s
<code>find_most_complex_functions</code>	2 high-risk functions identified (complexity 28 and 17) became refactoring stories	Manual code review looking for long functions — subjective, likely wouldn't identify the right ones	~1-2h → 5s (and more accurate)
<code>find_dead_code</code>	80 uncalled symbols → housekeeping story with precise scope	Would never be done in a retrospective. "We should clean up code" — vague, no action	Infinite → 5s (made it possible)
<code>find_central_symbols</code>	Most-connected type (62 references) confirmed analysis focus was correct	Impossible without graph analysis. Guessing based on code familiarity	Infeasible → 5s
<code>get_symbol_context</code>	Engine's main function: 7 callees, detected community, mapped flow	Read 7 files, trace imports, understand dependencies manually	~20 min → 5s

Gains by Cycle Phase

BMAD Phase	Without Memtrace	With Memtrace
Planning	Epics based on intuition about architecture	Epics validated against real modules detected by the graph
Implementation	Edit function without knowing blast radius — discover in PR that it broke 6 files	<code>get_impact</code> pre-edit: HIGH/CRITICAL risk flagged before writing code
Code Review	Review diff without context of what's outside it	<code>get_impact</code> reveals indirectly affected files; <code>find_dependency_path</code> finds edge cases
Testing	"Tests seem to cover everything"	<code>list_processes</code> → cross-reference with E2E → "3 user flows without coverage"
Retrospective	Qualitative discussion: "that component feels large"	Quantitative discussion: "complexity 17, 230 lines, 80 dead symbols — here are 7 housekeeping stories"

Before × After: Action Structure

Aspect	Before (without Memtrace)	After (with Memtrace)
Tech debt	1 vague entry in <code>deferred-work.md</code>	25 items catalogued by severity, origin, and owner
Refactoring	"We should refactor component X"	Story H1: <code>updateEngine</code> (complexity 28 → target < 15) with precise scope and files
Housekeeping	"There's dead code somewhere"	Story H3: 80 symbols to remove, exact file list
Product decisions	Discussion postponed "for later"	5 pending decisions catalogued as <code>decision-pending</code> with traceable origin
CI/CD	Monolithic pipeline, slow feedback	Parallelizable jobs, complexity and dead code gates, deterministic seed

What Was Previously Impossible

Insight	Tool
Know that an interface has 62 references and is the system's most central symbol	<code>find_central_symbols</code>
Discover that the rule engine belongs to a logical community	<code>get_symbol_context</code> / <code>list_communities</code>
Precisely quantify an epic's scope in symbols and edges	<code>get_evolution(compound)</code>
Identify uncalled functions without exhaustive manual grep	<code>find_dead_code</code>

💡 **Conclusion:** Memtrace didn't replace qualitative analysis — it gave it **quantitative teeth**. Where there was "that module seems large", there's now "cognitive complexity 28, 95 lines, 6 callees". This turned subjective discussions into measurable action items.

1. Memtrace Tools Summary

Tool	What it does	Best moment
<code>get_codebase_briefing</code>	Overview: scale, modules, risks, dead code	Start of any session
<code>find_code</code>	Semantic code search ("retry logic", "auth token")	Exploring code without knowing exact names
<code>find_symbol</code>	Exact symbol name lookup	When you know what to find
<code>get_symbol_context</code>	Callers + callees + community + process for a symbol	Before editing any function

Tool	What it does	Best moment
<code>get_impact</code>	Blast radius: what would break if I change X?	Before refactoring, before merge
<code>find_most_complex_functions</code>	Top-N most complex functions	Prioritize refactoring, PR gate
<code>find_dead_code</code>	Functions/methods with zero callers	Cleanup, PR gate
<code>find_central_symbols</code>	Highest PageRank symbols (most connected)	Identify load-bearing code
<code>find_bridge_symbols</code>	Architectural chokepoints between subsystems	High-risk refactoring
<code>find_dependency_path</code>	Shortest path between symbol A and B	Debugging, dependency analysis
<code>get_evolution</code>	What changed between two dates/commits	Retrospective, pre-PR check
<code>get_changes_since</code>	What changed since my last session	Session continuity
<code>get_process_flow</code>	Step-by-step execution flow	Test design, debugging
<code>list_processes</code>	All detected execution flows	Test coverage mapping
<code>list_communities</code>	Logical modules detected by Louvain	Architecture analysis
<code>get_api_topology</code>	Who calls which endpoint (cross-service)	Cross-service dependency analysis
<code>get_cochange_context</code>	Which files historically change together?	Hidden coupling
<code>get_timeline</code>	Complete symbol history over time	Debugging, "when did this break?"

2. Integration by BMAD Workflow

Each section shows: the BMAD workflow, the target agent (with the command to invoke them), the Memtrace tool, a short prompt, and the value.

2.1 Planning (PRD → Epics → Stories)

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
<code>bmad-create-epics-and-stories</code>	John — PM (<code>/bmad-agent-pm</code>)	<code>get_codebase_briefing</code>	"Before creating epics, run <code>get_codebase_briefing</code> and confirm the epics map to the actual code modules."	Confirms epics map to real modules
<code>bmad-create-story</code>	John (<code>/bmad-agent-pm</code>) or Amelia (<code>/bmad-agent-dev</code>)	<code>get_symbol_context</code>	"For each dependency listed in the epic, run <code>get_symbol_context</code> and populate 'Previous Story Intelligence' with real callers/callees."	Populates dependencies with real graph data

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-check-implementation-readiness	Winston — Architect (/bmad-agent-architect)	get_impact	"Run <code>get_impact</code> on each story's target. If HIGH/CRITICAL, mark as blocker."	Detects if story touches high-risk code

2.2 Implementation (dev-story)

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
Story start	Amelia — Dev (/bmad-agent-dev)	find_code	"Use <code>find_code</code> with natural language to locate existing code related to this feature before starting implementation."	Finds patterns faster than glob/grep
Before editing ANY file	Amelia (/bmad-agent-dev)	get_impact	"Before editing this function, run <code>get_impact(direction='both')</code> . If HIGH/CRITICAL risk, warn me and suggest a safer approach."	Safety gate before edits
During implementation	Amelia (/bmad-agent-dev)	find_code	"Use <code>find_code</code> to search for existing patterns on how we handle [Zod/RLS/geocoding] before implementing from scratch."	Pattern reuse
After implementing	Amelia (/bmad-agent-dev)	get_evolution	"After implementing, run <code>get_evolution(from='<timestamp>', mode='compound')</code> to check for unintended changes."	Detects regressions
For APIs	Amelia (/bmad-agent-dev)	find_api_endpoints + find_api_calls	"Run <code>find_api_endpoints</code> and <code>find_api_calls</code> to see who calls this endpoint before changing the signature."	Avoids breaking consumers

2.3 Code Review

Layer	Agent (command)	Memtrace Tool	Short Prompt	Why
Blind Hunter	Amelia — Dev (/bmad-code-review)	get_impact	"For each modified function, run <code>get_impact</code> and list downstream files not in the diff — review them too."	Files affected outside the diff
Edge Case Hunter	Amelia (/bmad-code-	find_dependency_path	"Run <code>find_dependency_path</code> between modules unrelated to the diff. Paths	Surprise paths =

Layer	Agent (command)	Memtrace Tool	Short Prompt	Why
	review)		found = edge cases."	edge cases
Acceptance Auditor	Amelia (/bmad-code-review)	get_process_flow	"Run get_process_flow on the changed flow. Each step = one AC. Verify all are covered by tests."	Each process step = one AC
Automated gate (CI)	N/A (pipeline)	find_dead_code + find_most_complex_functions	"Add a CI step that compares find_dead_code and find_most_complex_functions on branch vs main. Fail on regression."	Automated quality

2.4 Testing

Activity	Agent (command)	Memtrace Tool	Short Prompt	Why
Coverage analysis	Murat — QA (/bmad-tea)	list_processes	"Run list_processes and cross-reference with e2e/ specs. List user flows without E2E coverage."	Which flows lack E2E?
Test design	Murat (/bmad-tea)	get_process_flow	"For process [Name], run get_process_flow . Each step in the flow should generate at least one test case."	Each step = one test case
Gap analysis	Murat (/bmad-tea)	find_symbol	"List exported functions in modified files with find_symbol . Check if each has test coverage."	Exported functions without tests = gap
Edge case discovery	Murat (/bmad-tea)	get_impact	"Run get_impact(direction='upstream') on entry functions. Callers with unexpected inputs = new edge cases."	Unexpected inputs from callers

2.5 Retrospective

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
Epic overview	Amelia — Dev (/bmad-retrospective)	get_evolution	"Run get_evolution(mode='compound') from epic start to end to quantify scope."	Quantified scope
What to refactor	Amelia (/bmad-retrospective)	find_most_complex_functions	"Run find_most_complex_functions(top_n=15) and suggest refactoring stories for the highest-risk ones."	Objective pain point list

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
What to clean up	Amelia (/bmad-retrospective)	find_dead_code	"Run find_dead_code(limit=30) . Turn the candidates into a housekeeping story."	Quantified cleanup scope
What is critical	Amelia (/bmad-retrospective)	find_central_symbols	"Run find_central_symbols(limit=15) . Mark as load-bearing — extra care when refactoring."	Structurally critical code

2.6 Sprint Planning

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
Prioritization	Amelia — Dev (/bmad-sprint-planning)	find_most_complex_functions	"Run find_most_complex_functions and prioritize stories touching high-complexity functions."	Complex functions = higher risk
Estimation	Amelia (/bmad-sprint-planning)	get_impact	"Run get_impact on each story's targets. Large blast radius = more effort."	Blast radius as effort proxy
Ordering	Amelia (/bmad-sprint-planning)	get_codebase_briefing	"Run get_codebase_briefing and validate whether story order makes architectural sense."	Order validated by real architecture

2.7 Course Correction

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
Understand impact	Amelia — Dev (/bmad-correct-course)	get_evolution	"Run get_evolution since sprint start to understand what was already built."	Progress baseline
Change risk	Amelia (/bmad-correct-course)	get_impact	"Run get_impact on new targets. If the change breaks existing code, suggest mitigation."	Risk assessment
Rollback	Amelia (/bmad-correct-course)	get_timeline	"Run get_timeline on affected symbols for full history in case rollback is needed."	History for reversion

2.8 Debugging / Incident

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
Find cause	Amelia — Dev (/bmad-quick-dev)	find_code	"Use <code>find_code</code> with the error message '...' to locate where the error is generated in code."	Locates error origin
When it broke	Amelia (/bmad-quick-dev)	get_timeline	"Run <code>get_timeline</code> on symbol [name] to see all changes and identify when the bug was introduced."	Complete symbol history
Blast radius	Amelia (/bmad-quick-dev)	get_impact	"Run <code>get_impact(symbol, direction='downstream')</code> on the broken symbol to see what else was affected."	What else broke
Hidden coupling	Amelia (/bmad-quick-dev)	get_cochange_context	"Run <code>get_cochange_context</code> on the broken file. Files that always change together may be related."	Hidden dependencies

2.9 Session Continuity

Moment	Agent (command)	Memtrace Tool	Short Prompt	Why
First session	Any BMAD agent	get_codebase_briefing	"Run <code>get_codebase_briefing(detail_level='full')</code> to understand the architecture before starting."	Architecture in 30s
Resume session	Any BMAD agent	get_changes_since	"Run <code>get_changes_since</code> on the repo to see what changed since the last session."	Quick context
Understand branch	Any BMAD agent	get_codebase_briefing	"Run <code>get_codebase_briefing</code> on the current branch to understand what was built on it."	Branch scope

2.10 Story & Epic Creation

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-create-epics-and-stories	John — PM (/bmad-agent-pm)	get_codebase_briefing	"Before creating epics, run <code>get_codebase_briefing(detail_level='full')</code> and confirm the epics map to the actual code modules."	Confirms epics map to real modules
bmad-create-epics-and-stories	John (/bmad-agent-pm)	list_communities	"Run <code>list_communities</code> and validate that epic boundaries align with detected logical modules. Epics spanning many communities = candidates to split."	Architectural alignment

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-create-epics-and-stories	John (/bmad-agent-pm)	find_most_complex_functions	"Run <code>find_most_complex_functions(top_n=10)</code> . Surface high-risk areas any story should account for in its risk section."	Proactive risk awareness
bmad-create-story	John (/bmad-agent-pm) or Amelia (/bmad-agent-dev)	get_symbol_context	"For each dependency listed in the epic, run <code>get_symbol_context</code> and populate 'Previous Story Intelligence' with real callers/callees."	Populates dependencies with real graph data
bmad-create-story	John or Amelia	get_impact	"Run <code>get_impact</code> on each story's primary target. If HIGH/CRITICAL, mark as risk in the story file (not just verbal warning)."	Risk-aware story creation

2.11 Architecture & Readiness

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-create-architecture	Winston — Architect (/bmad-agent-architect)	list_communities	"Run <code>list_communities</code> . Compare the proposed architecture with detected logical modules. If the new design splits an existing community, surface that explicitly."	Architectural alignment
bmad-create-architecture	Winston (/bmad-agent-architect)	get_api_topology	"Run <code>get_api_topology</code> to map cross-service dependencies. New architecture must account for existing service boundaries."	Cross-service awareness
bmad-create-architecture	Winston (/bmad-agent-architect)	find_bridge_symbols	"Run <code>find_bridge_symbols</code> . Identify architectural chokepoints before designing around them."	Chokepoint awareness
bmad-check-implementation-readiness	Winston (/bmad-agent-architect)	get_impact	"Run <code>get_impact</code> on each story's target. If HIGH/CRITICAL, mark as blocker — story must include mitigation or be re-scoped."	Detects if story touches high-risk code
bmad-check-implementation-readiness	Winston (/bmad-agent-architect)	get_api_topology	"Run <code>get_api_topology</code> for any story that modifies an endpoint. Unverified consumers = readiness blocker."	Consumer impact verification

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-check-implementation-readiness	Winston (/bmad-agent-architect)	list_communities	"Run <code>list_communities</code> and validate stories don't fragment a community without explicit refactoring plan."	Community integrity check

2.12 Documentation & Project Context

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-document-project	Paige — Tech Writer (/bmad-agent-tech-writer)	get_codebase_briefing	"Run <code>get_codebase_briefing(detail_level='full')</code> to understand the architecture before documenting. Don't guess the structure — use the graph."	Architecture in 30s replaces hours of manual exploration
bmad-document-project	Paige (/bmad-agent-tech-writer)	list_communities	"Run <code>list_communities</code> and use detected logical modules as the document outline. Each community = one documentation section."	Structure from data, not intuition
bmad-document-project	Paige (/bmad-agent-tech-writer)	list_processes	"Run <code>list_processes</code> to enumerate execution flows. Each flow deserves documentation of its sequence and entry points."	Flow-based documentation
bmad-document-project	Paige (/bmad-agent-tech-writer)	find_central_symbols	"Run <code>find_central_symbols(limit=15)</code> and give extra documentation weight to load-bearing code."	Focus docs on what matters most
bmad-generate-project-context	Amelia — Dev (/bmad-agent-dev)	get_codebase_briefing	"Run <code>get_codebase_briefing(detail_level='full')</code> — use the graph data as the primary source for the context file, NOT file-system walking."	Project-context.md driven by real architecture
bmad-generate-project-context	Amelia (/bmad-agent-dev)	list_communities	"Run <code>list_communities</code> to populate the module breakdown section of project-context.md."	Modules from graph, not folder guesses

2.13 E2E Test Generation

BMAD Workflow	Agent (command)	Memtrace Tool	Short Prompt	Why
bmad-qa-generate-e2e-tests	Murat — QA (/bmad-tea)	list_processes	"Run <code>list_processes</code> to enumerate all execution flows. Each flow = a candidate for E2E test generation."	Which flows need tests?
bmad-qa-generate-e2e-tests	Murat (/bmad-tea)	get_process_flow	"For each process, run <code>get_process_flow(name)</code> . Each step in the flow should generate at least one E2E test case (happy path + relevant negatives)."	Each step = one test case
bmad-qa-generate-e2e-tests	Murat (/bmad-tea)	find_api_endpoints	"Run <code>find_api_endpoints</code> to list all endpoints. Cross-reference with existing test files — uncovered endpoints = generation priority."	Full API surface coverage

2.14 Technical Research (Conditional)

⚠️ CONDITIONAL USE: Memtrace is only relevant here when the research topic involves the **current codebase itself** (e.g., "what ORM would replace our current Prisma setup?", "should we refactor this module to use EventEmitter?"). If the research is purely external (e.g., "React vs Vue for a new project"), skip ALL Memtrace tools — they add no value.

BMAD Workflow	Agent (command)	Memtrace Tool	Condition	Short Prompt	Why
bmad-technical-research	Any agent	find_code	ONLY if topic is about current code	"Use <code>find_code</code> with the research topic keywords to locate relevant existing code that would be affected by or replaced by the technology being researched."	Finds affected code
bmad-technical-research	Any agent	get_symbol_context	ONLY if specific symbols identified	"Run <code>get_symbol_context</code> on symbols that would be impacted by the proposed technology change."	Impact analysis
bmad-technical-research	Any agent	find_most_complex_functions	ONLY if evaluating refactoring options	"Run <code>find_most_complex_functions(top_n=10)</code> to identify candidate areas where the new technology would provide the most benefit."	Prioritize by complexity
bmad-technical-research	Any agent	get_impact	ONLY if evaluating migration effort	"Run <code>get_impact</code> on symbols that would need migration. Large blast radius = higher cost estimate."	Effort estimation

2.15 Complete Prompts by Workflow

Copy and paste the prompt below to the indicated agent. Each prompt already includes all recommended Memtrace tools for that workflow, in the correct order, with safety triggers.

Epic & Story Planning

Agent: John — PM (/bmad-agent-pm)

Before creating epics and stories, run these Memtrace checks:

1. `get_codebase_briefing(detail_level="full")` – understand current architecture
2. `find_most_complex_functions(top_n=10)` – identify high-risk code
3. `list_communities` – confirm detected modules match proposed epic boundaries

Then, for each created story:

4. `get_symbol_context` on listed dependencies – populate "Previous Story Intelligence" with real graph callers/callees
5. `get_impact` on the primary target – if HIGH/CRITICAL, mark as risk in the story file

If uncertain about story ordering:

6. `find_dependency_path` between targets of consecutive stories to validate precedence

Story Implementation

Agent: Amelia — Dev (/bmad-agent-dev)

You are implementing this story. Follow this Memtrace protocol:

BEFORE editing any file:

1. `find_code` with natural language about the feature – locate existing related code
2. `get_impact` on the primary target (`direction="both"`) – if HIGH/CRITICAL, warn me before proceeding
3. `get_symbol_context` on each listed dependency – understand real callers/callees

DURING implementation:

4. `find_code` to search existing patterns ("how do we handle Zod validation?", "is there an RLS helper?")
5. `find_api_endpoints` + `find_api_calls` – before changing a Server Action signature, see who calls it
6. `find_bridge_symbols` – if the target is a bridge symbol, refactor with extra care

AFTER implementing:

7. `get_evolution(from="<timestamp before starting>", mode="compound")` – check for unintended changes
8. `find_dead_code` – confirm no new symbols without callers were introduced

Code Review (3 layers)

Agent: Amelia — Dev (/bmad-code-review)

Run adversarial review in 3 layers with Memtrace support:

LAYER 1 – BLIND HUNTER:

1. `get_evolution` on the branch (`mode="compound"`) – baseline of what was touched
2. `get_impact` on EACH modified function – list downstream files NOT in the diff (must be reviewed too)
3. `find_bridge_symbols` on the diff – refactors on bridges = high risk, double scrutiny
4. `find_central_symbols` on modified files – if load-bearing symbols were touched, verify carefully

LAYER 2 – EDGE CASE HUNTER:

5. `find_dependency_path` between modules unrelated to the diff – paths found = edge cases
6. `get_impact(direction="upstream")` on entry functions – callers with unexpected inputs?

LAYER 3 – ACCEPTANCE AUDITOR:

7. `get_process_flow` of the changed flow – each step = one AC, verify test coverage
8. `find_dead_code` – did the PR introduce new symbols without callers?
9. `find_most_complex_functions` – compare branch vs main, fail if a function with complexity > 15 was introduced

 QA & Testing**Agent:** Murat — QA (`/bmad-tea`)

Analyze project test quality using Memtrace:

1. `list_processes` – map all detected execution flows
2. Cross-reference with `e2e/*.spec.ts` – list flows WITHOUT E2E coverage
3. For each uncovered flow: `get_process_flow("[name]")` – each step = one test case to generate
4. `find_symbol` on modified files – list exported functions, check if each has a test
5. `get_impact(direction="upstream")` on entry functions – callers with unexpected inputs = new edge cases
6. `find_dead_code` – verify tests don't reference functions that no longer exist

 Epic Retrospective**Agent:** Amelia — Dev (`/bmad-retrospective`)

Run the retrospective with quantitative Memtrace analysis:

1. `get_evolution(mode="compound", from="<epic start date>")` – quantified scope: symbols, edges, episodes
2. `find_most_complex_functions(top_n=15)` – prioritize refactoring by highest risk
3. `find_dead_code(limit=30)` – turn candidates into a housekeeping story
4. `find_central_symbols(limit=15)` – mark load-bearing, extra care when refactoring
5. `list_communities` – compare structure before/after the epic, detect god-modules
6. `get_codebase_briefing` – compare metrics (nodes, edges) with pre-epic briefing if available

 Sprint Planning**Agent:** Amelia — Dev (`/bmad-sprint-planning`)

Plan the sprint with Memtrace data:

1. `get_codebase_briefing` – overview of current architecture
2. `find_most_complex_functions(top_n=15)` – stories touching complex functions = priority
3. `get_impact` on each candidate story's targets – large blast radius = more effort, adjust estimates
4. `list_processes` – understand full functional scope when prioritizing
5. `find_dependency_path` between consecutive stories – validate no cyclic dependencies in ordering

 Course Correction (mid-sprint change)

Agent: Amelia — Dev (/bmad-correct-course)

Assess the impact of the proposed change with Memtrace:

1. `get_evolution(from="<sprint start>", mode="compound")` – what was already built?
2. `get_impact` on new targets – does the change break existing code? If so, suggest mitigation
3. `get_timeline` on affected symbols – full history for rollback if needed
4. `find_dependency_path` between the change and critical code – indirect impact paths
5. `get_cochange_context` on affected files – which files always change together? Do they also need adjustment?

Debugging / Incident Investigation

Agent: Amelia — Dev (/bmad-quick-dev)

Investigate the issue with Memtrace:

1. `find_code` with the error message `"..."` – locate where the error is generated
2. `get_timeline` on the error symbol – when was the last change? What changed?
3. `get_impact(symbol, direction="downstream")` – what else broke?
4. `get_process_flow` of the affected flow – trace the full path, find the failing step
5. `get_cochange_context` on the broken file – files that change together may be related
6. `get_evolution(from="<last stable version>")` – what changed since it worked?

Session Start or Resume

Agent: Any (/bmad-help or target agent)

Prepare session context with Memtrace:

1. `get_codebase_briefing(detail_level="full")` – understand architecture in 30 seconds
2. `get_changes_since` – what changed since last session? (if resuming)
3. `list_communities` – view of logical modules
4. `find_central_symbols(limit=10)` – know which symbols are load-bearing before touching anything

Epic Creation

Agent: John — PM (/bmad-agent-pm)

Before creating epics, run these Memtrace checks:

1. `get_codebase_briefing(detail_level="full")` – understand current architecture before proposing epics
2. `list_communities` – validate that proposed epic boundaries map to detected logical modules (Louvain communities)
3. `find_most_complex_functions(top_n=10)` – surface high-risk code areas; any epic touching them inherits risk

Then for each epic created:

4. For each story dependency listed: `get_symbol_context(symbol)` – populate "Previous Story Intelligence" with real graph
5. `get_impact` on each story's primary target – if HIGH/CRITICAL, mark the story file with an explicit risk flag and req

Validation:

6. find_dependency_path between targets of consecutive stories – validate story ordering before locking the sprint plan
7. confirm no epic spans more than 3 communities without explicit justification

Story Creation (Individual)

Agent: John — PM (/bmad-agent-pm) or Amelia — Dev (/bmad-agent-dev)

Create this story with graph-backed intelligence:

1. get_codebase_briefing(detail_level="summary") – refresh architecture context
2. get_symbol_context on each dependency listed in the story – populate "Previous Story Intelligence" with real callers
3. get_impact on the story's primary target – if HIGH or CRITICAL:
 - Mark the story file with "[Risk: HIGH]" or "[Risk: CRITICAL]"
 - Add affected files list
 - Require architect sign-off before dev-story
4. find_dependency_path from this story's target to known load-bearing symbols (from find_central_symbols) – indirect co
5. After story file is written: validate that ALL acceptance criteria trace to at least one process step from get_proce

Architecture Design

Agent: Winston — Architect (/bmad-agent-architect)

Design architecture backed by real code structure:

1. list_communities – baseline of current logical modules
2. get_api_topology – current cross-service dependency map
3. find_bridge_symbols – identify current architectural chokepoints (betweenness centrality)

When proposing architecture changes:

4. Compare proposed module boundaries against list_communities output – if the new design splits an existing community,
5. Run get_impact on any symbol that would move between modules – blast radius = migration cost
6. Identify find_bridge_symbols that would be created or removed by the new architecture – bridges are high-maintenance

Validation:

7. Every new module boundary must have a justification against community data
8. Cross-service dependencies from get_api_topology must not increase without explicit benefit

Implementation Readiness Check

Agent: Winston — Architect (/bmad-agent-architect)

Validate readiness with Memtrace data:

1. list_communities – confirm current architecture matches the planned one
2. get_api_topology – map all cross-service dependencies

For each story in the sprint:

3. get_impact on the story's primary target – HIGH or CRITICAL risk = BLOCKER:
 - Story must include mitigation plan or be re-scoped
 - Document the risk in the readiness report

4. If the story modifies an endpoint: `get_api_topology` filtered to that endpoint → verify ALL consumers were accounted
5. `find_bridge_symbols` – if any story refactors a bridge symbol, flag for architect-led design review

Output: readiness report with (a) go/no-go per story, (b) blocker list with graph evidence, (c) risk matrix based on ge

Document Project

Agent: Paige — Tech Writer (`/bmad-agent-tech-writer`)

Document this project using graph data as the primary source:

1. `get_codebase_briefing(detail_level="full")` – architecture overview: scale, modules, risks, dead code
2. `list_communities` – use detected logical modules as the documentation outline. Each community = one section
3. `list_processes` – enumerate all execution flows. Each flow deserves documentation of its sequence and entry points
4. `find_central_symbols(limit=15)` – give extra documentation weight and detail to load-bearing code (high PageRank)
5. `find_most_complex_functions(top_n=10)` – document these with extra care; complex functions need clear contracts and e

Documentation structure rule:

- Each community from `list_communities` → one top-level doc section
- Each process from `list_processes` → one flow-documentation subsection
- `find_central_symbols` results → expanded API/contract documentation with callers and callees from `get_symbol_context`
- `find_dead_code` results → explicitly NOT documented (reduce noise – dead code is not user-facing)

Generate Project Context

Agent: Amelia — Dev (`/bmad-agent-dev`)

Generate `project-context.md` driven entirely by Memtrace graph data:

DO NOT walk the filesystem with `tree/ls/glob`. Use Memtrace as the single source of truth.

1. `get_codebase_briefing(detail_level="full")` – this IS your outline:
 - Symbol counts → project scale section
 - Community list → module breakdown section
 - Top risks → risk section
2. `list_communities` – populate the "Modules" section with each community name + its top symbols from `find_central_symbols`
3. `list_processes` – populate the "Key Flows" section with each process name + entry point
4. `find_central_symbols(limit=10)` – add a "Load-Bearing Code" section listing these symbols with their PageRank context
5. `find_most_complex_functions(top_n=10)` – add a "Complexity Hotspots" section

Structure of `project-context.md`:

- Scale & Architecture (from `get_codebase_briefing`)
- Modules (from `list_communities`)
- Key Flows (from `list_processes`)
- Load-Bearing Code (from `find_central_symbols`)
- Complexity Hotspots (from `find_most_complex_functions`)
- Risk Areas (from `get_codebase_briefing` risk summary)

 Generate E2E Tests**Agent:** Murat — QA (/bmad-tea)

Generate E2E tests driven by detected execution flows:

1. `list_processes` – enumerate ALL detected execution flows in the codebase
2. Cross-reference with existing `e2e/*.spec.ts` files – list flows WITHOUT E2E coverage as PRIORITY targets

For each uncovered flow:

3. `get_process_flow("[process name]")` – trace the full execution path step by step

Test generation rules:

4. Each step in `get_process_flow` output = at least one E2E test case (happy path + relevant negative paths)
5. Each test must reference the exact function name and file:line from the flow step
6. `find_api_endpoints` – cross-reference: every endpoint involved in the flow must have its HTTP method and path verified

Quality checks after generation:

7. `find_dead_code` – verify no generated test references a function that no longer exists (orphan test)
8. `find_most_complex_functions` – ensure complexity hotspots from the flow have extra assertion depth

Output: a test file per process with (a) flow diagram comment, (b) per-step test cases, (c) coverage of positive + negative paths

 Technical Research (Conditional)**Agent:** Any relevant agent BEFORE USING MEMTRACE: Determine if the research topic involves the CURRENT codebase.

IF the research is about:

- External technology comparison ("React vs Vue") → SKIP all Memtrace tools entirely
- Evaluating a library/framework for a NEW project → SKIP all Memtrace tools
- General technology trends, ecosystem analysis → SKIP all Memtrace tools

IF the research is about:

- Replacing/upgrading technology IN the current codebase → USE Memtrace
- Refactoring approach evaluation on current code → USE Memtrace
- Migration effort estimation for current codebase → USE Memtrace

When applicable, use these Memtrace tools:

1. `find_code` with research topic keywords – locate existing code affected by or replaceable by the technology being researched
2. `get_symbol_context` on identified symbols – understand their role before proposing changes
3. `find_most_complex_functions(top_n=10)` – if evaluating refactoring approaches, prioritize by complexity
4. `get_impact` on symbols that would need migration – large blast radius = higher cost, longer timeline
5. `find_dependency_path` between current implementation and proposed replacement – hidden dependency chains = extra migration
6. `get_cochange_context` on affected files – files that historically change together may need coordinated migration

Output must include: (a) affected symbols count, (b) blast radius from `get_impact`, (c) estimated effort tier (Small/Medium/Large)

3. AI-Guided Setup

 **HUMAN INSTRUCTIONS:** This section is a script for the AI to execute. Open BMAD in your project, paste the link to this guide, and say: **"Run the guided setup from section 3."** The AI will ask you tool by tool whether you want to install each one.

 LLM INSTRUCTION BLOCK — START

YOU (the LLM) are executing an interactive guided setup. Your goal: walk the user through each Memtrace tool and ask if they want to integrate it into the corresponding BMAD workflow. At the end, update the BMAD agent/workflow files to incorporate all accepted tools.

Setup Protocol:

- 1. Greet the user in their language. Explain: "I'll walk you through each Memtrace tool. For each one, I'll explain what it does, where it fits in the BMAD workflow, which agent to talk to (with the command to invoke them), and a short prompt to ask them. You answer **Y** (Yes) or **N** (No). At the end, I'll update your BMAD configs automatically."
- 2. For each of the 15 tools below, present the tool description, then ask for confirmation. Track all "Y" answers.
- 3. After going through all tools, proceed to **Phase 2: Apply** below.

Tool 1: `get_codebase_briefing`

What it does: Codebase overview — scale, modules, risks, dead code.

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-create-epics-and-stories</code>	John — PM (<code>/bmad-agent-pm</code>)	"Before creating epics, run <code>get_codebase_briefing</code> and confirm the epics map to the actual code modules."
<code>bmad-sprint-planning</code>	Amelia — Dev (<code>/bmad-sprint-planning</code>)	"Run <code>get_codebase_briefing</code> and validate whether story order makes architectural sense."
Session start (any)	All agents	"Run <code>get_codebase_briefing(detail_level='full')</code> to understand the architecture before starting."

Question: Install `get_codebase_briefing` in the above workflows? (Y/N)

Tool 2: `find_code`

What it does: Semantic code search ("retry logic", "auth token pooling").

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-dev-story</code>	Amelia — Dev (<code>/bmad-agent-dev</code>)	"Use <code>find_code</code> with natural language to locate existing code related to this feature before starting."

Workflow	Agent (command)	Prompt
Debugging	Amelia — Dev (/bmad-quick-dev)	"Use <code>find_code</code> with the error message to locate where the error is generated."

Question: Install `find_code` as the primary search tool in dev-story and debugging? (Y/N)

Tool 3: `find_symbol`

What it does: Exact symbol name lookup (function, class, type).

Possible integrations:

Workflow	Agent (command)	Prompt
bmad-dev-story	Amelia — Dev (/bmad-agent-dev)	"Use <code>find_symbol</code> to locate the exact implementation of the referenced function/type."
QA	Murat — QA (/bmad-tea)	"List exported functions with <code>find_symbol</code> . Check if each has a corresponding test."

Question: Install `find_symbol` in dev-story and QA? (Y/N)

Tool 4: `get_symbol_context`

What it does: Callers, callees, community, and process for a symbol — 360° view.

Possible integrations:

Workflow	Agent (command)	Prompt
bmad-create-story	John — PM (/bmad-agent-pm)	"For each dependency listed in the epic, run <code>get_symbol_context</code> and populate 'Previous Story Intelligence' with real callers/callees."
bmad-dev-story	Amelia — Dev (/bmad-agent-dev)	"Before editing this function, run <code>get_symbol_context</code> to understand the full context."

Question: Install `get_symbol_context` in create-story and dev-story? (Y/N)

Tool 5: `get_impact`

What it does: Blast radius — how many symbols would be affected if X changes, with risk level.

Possible integrations:

Workflow	Agent (command)	Prompt
bmad-dev-story (pre-edit)	Amelia — Dev (/bmad-agent-dev)	"Before editing this function, run <code>get_impact(direction='both')</code> . If HIGH/CRITICAL, warn me."

Workflow	Agent (command)	Prompt
bmad-code-review	Amelia — Dev (/bmad-code-review)	"For each modified function, run <code>get_impact</code> . List downstream files not in the diff."
bmad-check-implementation-readiness	Winston — Architect (/bmad-agent-architect)	"Run <code>get_impact</code> on each story's target. HIGH/CRITICAL → blocker."
bmad-sprint-planning	Amelia — Dev (/bmad-sprint-planning)	"Run <code>get_impact</code> on targets. Large blast radius = more effort."
bmad-correct-course	Amelia — Dev (/bmad-correct-course)	"Run <code>get_impact</code> on new targets to assess change risk."

Question: Install `get_impact` as pre-edit safety gate + code review + readiness? (Y/N)

Tool 6: `find_most_complex_functions`

What it does: Top-N most complex functions (by cognitive complexity and out-degree).

Possible integrations:

Workflow	Agent (command)	Prompt
bmad-retrospective	Amelia — Dev (/bmad-retrospective)	"Run <code>find_most_complex_functions(top_n=15)</code> and suggest refactoring stories for the highest-risk ones."
bmad-sprint-planning	Amelia — Dev (/bmad-sprint-planning)	"Run <code>find_most_complex_functions</code> and prioritize stories touching high-complexity functions."
CI/CD gate	Pipeline (<code>.github/workflows/ci.yml</code>)	"Add a step that compares <code>find_most_complex_functions</code> on branch vs main. Fail if a new function exceeds threshold."

Question: Install `find_most_complex_functions` in retrospective, sprint-planning, and CI? (Y/N)

Tool 7: `find_dead_code`

What it does: Functions/methods with zero callers — removal candidates.

Possible integrations:

Workflow	Agent (command)	Prompt
bmad-retrospective	Amelia — Dev (/bmad-retrospective)	"Run <code>find_dead_code(limit=30)</code> . Turn the candidates into a housekeeping story."
bmad-code-review	Amelia — Dev (/bmad-code-review)	"Run <code>find_dead_code</code> on the branch. Flag if the PR introduced new symbols without callers."

Workflow	Agent (command)	Prompt
CI/CD gate	Pipeline (.github/workflows/ci.yml)	"Add a step that compares <code>find_dead_code</code> on branch vs main. Warn if it increases."

Question: Install `find_dead_code` in retrospective, code review, and CI? (Y/N)

Tool 8: `find_central_symbols`

What it does: Highest PageRank symbols — load-bearing code.

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-retrospective</code>	Amelia — Dev (<code>/bmad-retrospective</code>)	"Run <code>find_central_symbols(limit=15)</code> . Mark as load-bearing."
<code>bmad-dev-story</code>	Amelia — Dev (<code>/bmad-agent-dev</code>)	"Run <code>find_central_symbols</code> on the files you'll edit. Alert if it's a high-centrality symbol."

Question: Install `find_central_symbols` in retrospective and dev-story? (Y/N)

Tool 9: `find_bridge_symbols`

What it does: Architectural chokepoints — symbols connecting otherwise disconnected subsystems.

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-dev-story</code> (pre-refactor)	Amelia — Dev (<code>/bmad-agent-dev</code>)	"Before refactoring, run <code>find_bridge_symbols</code> . If the target is a bridge, alert — high risk."
<code>bmad-code-review</code>	Amelia — Dev (<code>/bmad-code-review</code>)	"Run <code>find_bridge_symbols</code> on the diff. Refactors on bridges = extra scrutiny."

Question: Install `find_bridge_symbols` as a pre-refactor alert? (Y/N)

Tool 10: `get_evolution`

What it does: What changed between two dates/commits — files and symbols.

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-retrospective</code>	Amelia — Dev (<code>/bmad-retrospective</code>)	"Run <code>get_evolution(mode='compound')</code> from epic start to end to quantify scope."

Workflow	Agent (command)	Prompt
bmad-dev-story (post-impl.)	Amelia — Dev (/bmad-agent-dev)	"After implementing, run <code>get_evolution</code> (from='<timestamp>', mode='compound') to detect unintended changes."
bmad-correct-course	Amelia — Dev (/bmad-correct-course)	"Run <code>get_evolution</code> since sprint start to understand progress."
bmad-code-review	Amelia — Dev (/bmad-code-review)	"Run <code>get_evolution</code> on the branch to see a baseline of what was touched."

Question: Install `get_evolution` in retrospective + dev-story + code review? (Y/N)

Tool 11: `get_changes_since`

What it does: What changed since my last session (session anchor).

Possible integrations:

Workflow	Agent (command)	Prompt
Session start (any)	All agents	"Run <code>get_changes_since</code> on the repo to see what changed since the last session."

Question: Install `get_changes_since` at the start of every BMAD session? (Y/N)

Tool 12: `get_process_flow`

What it does: Step-by-step execution flow (e.g., POST /login → auth → session → DB).

Possible integrations:

Workflow	Agent (command)	Prompt
QA	Murat — QA (/bmad-tea)	"For process [Name], run <code>get_process_flow</code> . Each flow step should generate at least one test case."
bmad-code-review (Acceptance Auditor)	Amelia — Dev (/bmad-code-review)	"Run <code>get_process_flow</code> on the changed flow. Each step = one AC. Verify coverage."
Debugging	Amelia — Dev (/bmad-quick-dev)	"Run <code>get_process_flow</code> to trace the full path of the failing request."

Question: Install `get_process_flow` in QA and Acceptance Auditor? (Y/N)

Tool 13: `list_processes`

What it does: All execution flows detected in the code.

Possible integrations:

Workflow	Agent (command)	Prompt
QA	Murat — QA (/bmad-tea)	"Run <code>list_processes</code> and cross-reference with <code>e2e/</code> specs. List flows without E2E coverage."
<code>bmad-sprint-planning</code>	Amelia — Dev (/bmad-sprint-planning)	"Run <code>list_processes</code> to map the full functional scope of the system."

Question: Install `list_processes` in QA and sprint-planning? (Y/N)

Tool 14: `list_communities`

What it does: Logical modules detected (Louvain community detection).

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-retrospective</code>	Amelia — Dev (/bmad-retrospective)	"Run <code>list_communities</code> and compare with folder structure. Detect god-modules."
<code>bmad-create-architecture</code>	Winston — Architect (/bmad-agent-architect)	"Run <code>list_communities</code> . Validate actual architecture matches the planned one."

Question: Install `list_communities` in retrospective and create-architecture? (Y/N)

Tool 15: `get_api_topology`

What it does: Who calls which endpoint — cross-service topology.

Possible integrations:

Workflow	Agent (command)	Prompt
<code>bmad-dev-story</code> (pre-API change)	Amelia — Dev (/bmad-agent-dev)	"Run <code>find_api_endpoints</code> and <code>find_api_calls</code> to see all consumers before changing the signature."
<code>bmad-check-implementation-readiness</code>	Winston — Architect (/bmad-agent-architect)	"Run <code>get_api_topology</code> to map cross-service dependencies."
<code>bmad-code-review</code>	Amelia — Dev (/bmad-code-review)	"Endpoint change? Run <code>get_api_topology</code> and verify all consumers."

Question: Install `get_api_topology` in dev-story and readiness check? (Y/N)

Phase 2: Scope — Per-Project or Global?

After all 15 tools have been processed, ask the user:

"Integrations will be installed via `bmad-customize`, which creates `.toml` override files — without modifying the original skills. This way, configurations survive BMAD updates."

"**Where to install?**" - **[P] Per-project** — files at `{project-root}/_bmad/custom/` (versioned in repo, affects only this project) - **[G] Global** — files at `{user-home}/.bmad/custom/` (affects ALL BMAD projects on this machine)

"**Recommendation:** Use [P] per-project, unless you want ALL your projects to use the same Memtrace rules."

Wait for answer. Store `install_scope` as `project` or `global`. Map to path: - `project` → `{project-root}/_bmad/custom/` - `global` → `{user-home}/.bmad/custom/`

Phase 3: Apply — 3-Level Installation Strategy

IMPORTANT: Do NOT edit the original `SKILL.md` files. Use ONLY `bmad-customize` (`.toml` override files). This guarantees integrations survive `bmad update`.

Install in 3 levels:

Preflight: Resolver Script

Before creating any override file, verify the customization resolver script exists. BMAD skills call this script on activation to merge base + team + user overrides. If the script is missing, overrides are silently ignored.

```
# Check if the script exists
test -f {project-root}/_bmad/scripts/resolve_customization.py || echo "MISSING"
```

If MISSING, create `{project-root}/_bmad/scripts/resolve_customization.py` with the resolver implementation. The script must: - Accept `--skill <skill-root>` and `--key <key>` arguments - Read `{skill-root}/customize.toml` (base), `{project-root}/_bmad/custom/{skill-name}.toml` (team), `{project-root}/_bmad/custom/{skill-name}.user.toml` (user) - Deep-merge: scalars override, tables deep-merge, arrays append (except arrays-of-tables keyed by `code` / `id` which merge by key) - Output the resolved section as JSON to stdout

Level 1: `activation_steps_prepend` — Tools that run AUTOMATICALLY on workflow start

Each agent receives a `.toml` file with `activation_steps_prepend`. These steps execute **without the user having to ask** — when they call `/bmad-code-review`, the agent automatically starts by running `get_evolution` and `find_most_complex_functions` before any manual analysis.

CRITICAL — Dual-section approach for agent persona skills: Skills of type: `agent` (personas like Murat, John, Winston, Amelia) have TWO sections in their `.toml` file. The framework resolves `--key agent` automatically; the agent additionally loads `--key workflow` via a SKILL.md activation step. This keeps methodology rules (`persistent_facts`) separate from execution steps (`activation_steps_prepend`).

- **Agent persona skills** (`bmad-agent-dev`, `bmad-agent-pm`, `bmad-agent-architect`, `bmad-agent-tech-writer`, `bmad-agent-analyst`, `bmad-agent-ux-designer`, `bmad-tea`) → use `[agent]` for `persistent_facts` + `[workflow]` for `activation_steps_prepend` (the SKILL.md On Activation step tells the agent to also load `--key workflow`)

- **Workflow skills** (bmad-dev-story , bmad-code-review , bmad-create-story , bmad-sprint-planning , bmad-retrospective , bmad-correct-course , bmad-quick-dev , bmad-create-epics-and-stories , bmad-create-architecture , bmad-check-implementation-readiness , bmad-document-project , bmad-generate-project-context , bmad-qa-generate-e2e-tests) → use [workflow] only (these are invoked as workflows and resolve --key workflow)

File format — Agent persona skills ([agent] + [workflow]):

```
# {install_scope_path}/bmad-{agent-name}.toml
# Methodology rules – resolved by framework via --key agent
[agent]
persistent_facts = [
    "BEFORE editing any function, run get_impact...",
]

# Execution steps – loaded by agent via SKILL.md step resolving --key workflow
[workflow]
activation_steps_prepend = [
    "Memtrace: execute get_codebase_briefing...",
    "Memtrace: execute find_code...",
    "Memtrace: execute get_changes_since...",
]
```

File format — Workflow skills ([workflow]):

```
# {install_scope_path}/bmad-{workflow-name}.toml
[workflow]
activation_steps_prepend = [
    "Memtrace: execute get_evolution on the current branch...",
    "Memtrace: execute find_most_complex_functions...",
]
persistent_facts = [
    "BEFORE editing any function, run get_impact...",
]
```

Agents and their automatic activations:

Agent-type skills use the dual-section approach: [agent] for methodology rules, [workflow] for execution steps. Workflow-type skills use [workflow] only. The "Sections" column shows which sections the .toml file contains.

File	Sections	Agent	Runs automatically on start
bmad-dev-story.toml	[workflow]	Amelia — Dev (/bmad-agent-dev)	get_codebase_briefing + find_code on the feature
bmad-agent-dev.toml	[agent] + [workflow]	Amelia — Dev (/bmad-agent-dev)	get_codebase_briefing + find_code on the feature + get_changes_since
bmad-code-review.toml	[workflow]	Amelia — Dev (/bmad-code-review)	get_evolution on branch + find_most_complex_functions vs main
bmad-retrospective.toml	[workflow]	Amelia — Dev (/bmad-retrospective)	get_evolution + find_most_complex_functions + find_dead_code + find_central_symbols

File	Sections	Agent	Runs automatically on start
<code>bmad-tea.toml</code>	<code>[agent]</code> + <code>[workflow]</code>	Murat — QA (<code>/bmad-tea</code>)	<code>list_processes</code> + <code>find_symbol</code> on exported functions
<code>bmad-sprint-planning.toml</code>	<code>[workflow]</code>	Amelia — Dev (<code>/bmad-sprint-planning</code>)	<code>get_codebase_briefing</code> + <code>find_most_complex_functions</code>
<code>bmad-correct-course.toml</code>	<code>[workflow]</code>	Amelia — Dev (<code>/bmad-correct-course</code>)	<code>get_evolution</code> since sprint start + <code>get_impact</code> on new targets
<code>bmad-agent-pm.toml</code>	<code>[agent]</code> + <code>[workflow]</code>	John — PM (<code>/bmad-agent-pm</code>)	<code>get_codebase_briefing</code> + <code>find_most_complex_functions</code>
<code>bmad-agent-architect.toml</code>	<code>[agent]</code> + <code>[workflow]</code>	Winston — Architect (<code>/bmad-agent-architect</code>)	<code>list_communities</code> + <code>get_api_topology</code>
<code>bmad-quick-dev.toml</code>	<code>[workflow]</code>	Amelia — Dev (<code>/bmad-quick-dev</code>)	<code>find_code</code> with the error message
<code>bmad-create-epics-and-stories.toml</code>	<code>[workflow]</code>	John — PM (<code>/bmad-agent-pm</code>)	<code>get_codebase_briefing</code> + <code>list_communities</code> + <code>find_most_complex_functions</code>
<code>bmad-create-story.toml</code>	<code>[workflow]</code>	John — PM or Amelia — Dev	<code>get_codebase_briefing</code> + <code>get_symbol_context</code> on dependencies
<code>bmad-create-architecture.toml</code>	<code>[workflow]</code>	Winston — Architect (<code>/bmad-agent-architect</code>)	<code>list_communities</code> + <code>get_api_topology</code> + <code>find_bridge_symbols</code>
<code>bmad-check-implementation-readiness.toml</code>	<code>[workflow]</code>	Winston — Architect (<code>/bmad-agent-architect</code>)	<code>get_impact</code> on each story target + <code>get_api_topology</code> + <code>list_communities</code>
<code>bmad-document-project.toml</code>	<code>[workflow]</code>	Paige — Tech Writer (<code>/bmad-agent-tech-writer</code>)	<code>get_codebase_briefing(detail_level='full')</code> + <code>list_communities</code> + <code>list_processes</code> + <code>find_central_symbols</code>
<code>bmad-generate-project-context.toml</code>	<code>[workflow]</code>	Amelia — Dev (<code>/bmad-agent-dev</code>)	<code>get_codebase_briefing(detail_level='full')</code> + <code>list_communities</code>
<code>bmad-qa-generate-e2e-tests.toml</code>	<code>[workflow]</code>	Murat — QA (<code>/bmad-tea</code>)	<code>list_processes</code> + <code>get_process_flow</code> on each uncovered process

⚠ `bmad-technical-research` is **conditional** — do NOT create a default `.toml` for it. The agent must evaluate the research topic first (see Section 2.14). Only if the topic involves the CURRENT codebase should Memtrace tools be used. This is implemented as a `persistent_fact` rule rather than `activation_steps_prepend`.

For each file listed above: create the corresponding `.toml` file with `activation_steps_prepend` + `persistent_facts` . Use the Sections column to determine whether the file uses `[workflow]` only or both `[agent]` + `[workflow]` .

SKILL.md activation step for agent-type skills: For each agent persona `.toml` file (dual-section), the agent's `SKILL.md` must include an "On Activation" step that resolves the `[workflow]` section at runtime. Add the following to the agent's `SKILL.md`: 2. `**Load workflow customizations**` – Run ``python _bmad/scripts/resolve_customization.py --key workflow --skill <this-skill-root-directory>`` (e.g., ``.agent/skills/{skill-name}/`` for OpenCode) and integrate the JSON output (``activation_steps_prepend``) as operational procedures into your context.

Level 2: `persistent_facts` — Methodology rules the agent considers THROUGHOUT the session

Reactive instructions — the agent loads them as context and applies them at the right moments (e.g., "BEFORE editing, `get_impact`", "DURING implementation, `find_code` for patterns"). Not rigid steps, but guidelines the agent follows as the flow unfolds.

Dual-section placement: For agent persona skills, `persistent_facts` go in `[agent]` (framework-resolved methodology rules) and `activation_steps_prepend` go in `[workflow]` (agent-loaded execution steps). For workflow skills, both stay in `[workflow]` .

Level 3: CI/CD gates (`.github/workflows/ci.yml`)

Only if the user accepted `find_most_complex_functions` and/or `find_dead_code` as CI gates: generate YAML config and offer to write to `.github/workflows/ci.yml` .

 **CI gates are generated but NOT applied automatically.** Review and adapt to the existing pipeline.

Phase 4: Summary

Print a summary table:

```

✓ INSTALLATION COMPLETE – {install_scope}

Files created at {install_scope_path}:

| File | Agent | Level |
|-----|-----|-----|
| bmad-dev-story.toml | Amelia – /bmad-agent-dev | 1 + 2 |
| bmad-agent-dev.toml | Amelia – /bmad-agent-dev | 1 + 2 |
| bmad-code-review.toml | Amelia – /bmad-code-review | 1 + 2 |
| bmad-retrospective.toml | Amelia – /bmad-retrospective | 1 + 2 |
| bmad-tea.toml | Murat – /bmad-tea | 1 + 2 |
| bmad-sprint-planning.toml | Amelia – /bmad-sprint-planning | 1 + 2 |
| bmad-correct-course.toml | Amelia – /bmad-correct-course | 1 + 2 |
| bmad-agent-pm.toml | John – /bmad-agent-pm | 1 + 2 |
| bmad-agent-architect.toml | Winston – /bmad-agent-architect | 1 + 2 |
| bmad-quick-dev.toml | Amelia – /bmad-quick-dev | 1 + 2 |
| bmad-create-epics-and-stories.toml | John – /bmad-agent-pm | 1 + 2 |
| bmad-create-story.toml | John or Amelia – /bmad-agent-pm or /bmad-agent-dev | 1 + 2 |
| bmad-create-architecture.toml | Winston – /bmad-agent-architect | 1 + 2 |
| bmad-check-implementation-readiness.toml | Winston – /bmad-agent-architect | 1 + 2 |
| bmad-document-project.toml | Paige – /bmad-agent-tech-writer | 1 + 2 |
```

```
| bmad-generate-project-context.toml | Amelia - /bmad-agent-dev | 1 + 2 |  
| bmad-qa-generate-e2e-tests.toml | Murat - /bmad-tea | 1 + 2 |
```

- 📄 Level 1 (activation_steps_prepend): Tools that run AUTOMATICALLY on workflow start
- 📄 Level 2 (persistent_facts): Instructions the agent considers throughout the session
- ⚠️ Level 3 (CI gates): Generated but NOT applied – review [.github/workflows/ci.yml](#)
- 🔬 bmad-technical-research: Conditional rule added – Memtrace used only when topic involves current codebase
- 🔗 Integrations survive `bmad update` (override files, not SKILL.md).
- 🗑️ To remove, delete the corresponding .toml file.

🤖 **LLM INSTRUCTION BLOCK — END**